ಭಾರತೀಯ ಮಾಹಿತಿ ತಂತ್ರಜ್ಞಾನ ಸಂಸ್ಥೆ ರಾಯಚೂರು
भारतीय सूचना प्रौद्योगिकी संस्थान रायचूर
Indian Institute of Information Technology Raichur

Secure Drone Communication System

CYBERSECURITY (AD310) — Assignment

Student: Rahul Yadav

CS23B1059

Date: April 2026

PART 1 — Python Source Code

Github Link of my Project: https://github.com/rahul6381yadav/cybersecurity

File: key_exchange.py

```
from typing import Tuple

from cryptography.hazmat.primitives.asymmetric import dh
from cryptography.hazmat.backends import default_backend
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.kdf.hkdf import HKDF


def generate_dh_keys():
    parameters = dh.generate_parameters(generator=2, key_size=2048,
backend=default_backend())
    private_key = parameters.generate_private_key()
    public_key = private_key.public_key()
    return private_key, public_key, parameters


def generate_shared_key(private_key, peer_public_key) -> bytes:
    return private_key.exchange(peer_public_key)
```

```python
def derive_session_keys(dh_shared_secret: bytes, aes_session_key: bytes) ->
Tuple[bytes, bytes]:
    """Derive (enc_key, mac_key) from DH shared secret + RSA-shared AES session key."""
    if not isinstance(dh_shared_secret, (bytes, bytearray)):
        raise TypeError("dh_shared_secret must be bytes")
    if not isinstance(aes_session_key, (bytes, bytearray)):
        raise TypeError("aes_session_key must be bytes")

    ikm = bytes(dh_shared_secret) + bytes(aes_session_key)
    hkdf = HKDF(
        algorithm=hashes.SHA256(),
        length=64,
        salt=None,
        info=b"drone-secure-protocol-v1",
    )
    okm = hkdf.derive(ikm)
    return okm[:32], okm[32:]
```

File: encryption.py

```python
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
import os


def _normalize_aes_key(key: bytes) -> bytes:
    if not isinstance(key, (bytes, bytearray)):
        raise TypeError("key must be bytes")
    key = bytes(key)
    if len(key) not in (16, 24, 32):
        raise ValueError("AES key must be 16/24/32 bytes")
    return key


def encrypt_data(key: bytes, plaintext):
    """Encrypt plaintext using AES-CTR.

    Returns (nonce, ciphertext). Nonce is 16 bytes.
    """
    key = _normalize_aes_key(key)

    if isinstance(plaintext, str):
        plaintext_bytes = plaintext.encode("utf-8")
    else:
        plaintext_bytes = bytes(plaintext)

    nonce = os.urandom(16)
    cipher = Cipher(algorithms.AES(key), modes.CTR(nonce))
```

```python
    encryptor = cipher.encryptor()
    ciphertext = encryptor.update(plaintext_bytes) + encryptor.finalize()
    return nonce, ciphertext


def decrypt_data(key: bytes, nonce: bytes, ciphertext: bytes) -> bytes:
    key = _normalize_aes_key(key)
    cipher = Cipher(algorithms.AES(key), modes.CTR(nonce))
    decryptor = cipher.decryptor()
    return decryptor.update(ciphertext) + decryptor.finalize()


def rsa_encrypt_key(rsa_pub, aes_key: bytes) -> bytes:
    """Encrypt an AES session key using RSA-OAEP(SHA-256)."""
    from cryptography.hazmat.primitives.asymmetric import padding
    from cryptography.hazmat.primitives import hashes

    if not isinstance(aes_key, (bytes, bytearray)):
        raise TypeError("aes_key must be bytes")

    return rsa_pub.encrypt(
        bytes(aes_key),
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None,
        ),
    )


def rsa_decrypt_key(rsa_priv, enc_key: bytes) -> bytes:
    """Decrypt an AES session key using RSA-OAEP(SHA-256)."""
    from cryptography.hazmat.primitives.asymmetric import padding
    from cryptography.hazmat.primitives import hashes

    return rsa_priv.decrypt(
        enc_key,
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None,
        ),
    )
```

File: authentication.py

```python
import hashlib
import os

def hash_password(password):
    salt = os.urandom(16)
    hashed = hashlib.sha256(salt + password.encode()).hexdigest()
    return salt, hashed

def verify_password(password, salt, stored_hash):
    return hashlib.sha256(salt + password.encode()).hexdigest() == stored_hash
```

File: signature.py

```python
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes

def generate_rsa_keys():
    private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
    public_key = private_key.public_key()
    return private_key, public_key

def sign_message(private_key, message):
    return private_key.sign(
        message,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )

def verify_signature(public_key, message, signature):
    public_key.verify(
        signature,
        message,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )
```

File: integrity.py

```python
import hmac
import hashlib

def generate_mac(key, message):
    return hmac.new(key, message, hashlib.sha256).digest()
```

```python
def verify_mac(key, message, mac):
    return hmac.compare_digest(generate_mac(key, message), mac)
```

File: replay.py

```python
import time

used_message_ids = set()


def generate_timestamp() -> int:
    return int(time.time())


def is_timestamp_in_window(timestamp: int, window_seconds: int = 30) -> bool:
    current = int(time.time())
    return abs(current - int(timestamp)) < int(window_seconds)


def is_replay(message_id) -> bool:
    return message_id in used_message_ids


def mark_timestamp_used(message_id) -> None:
    used_message_ids.add(message_id)


def reset_timestamps() -> None:
    used_message_ids.clear()
```

File: main.py

```python
import base64
import hashlib
import json
import os

from authentication import hash_password, verify_password
from encryption import decrypt_data, encrypt_data, rsa_decrypt_key, rsa_encrypt_key
from integrity import generate_mac, verify_mac
from key_exchange import derive_session_keys, generate_dh_keys, generate_shared_key
from replay import (
    generate_timestamp,
    is_replay,
    is_timestamp_in_window,
    mark_timestamp_used,
```

```python
    reset_timestamps,
)
from signature import generate_rsa_keys, sign_message, verify_signature


def _b64e(b: bytes) -> str:
    return base64.b64encode(b).decode("ascii")


def _b64d(s: str) -> bytes:
    return base64.b64decode(s.encode("ascii"))


def _canonical_bytes(obj) -> bytes:
    return json.dumps(obj, sort_keys=True, separators=(",", ":")).encode("utf-8")


def _signed_fields(packet: dict) -> dict:
    return {
        "drone_id": packet["drone_id"],
        "timestamp": packet["timestamp"],
        "nonce": packet["nonce"],
        "ciphertext": packet["ciphertext"],
        "enc_session_key": packet["enc_session_key"],
    }


def process_packet(packet: dict, *, server_rsa_priv, drone_sign_pub,
dh_shared_secret_server: bytes) -> bytes:
    if not is_timestamp_in_window(packet["timestamp"], window_seconds=30):
        raise ValueError("stale/out-of-window timestamp")

    aes_session_key = rsa_decrypt_key(server_rsa_priv,
_b64d(packet["enc_session_key"]))
    enc_key, mac_key = derive_session_keys(dh_shared_secret_server, aes_session_key)

    signed_bytes = _canonical_bytes(_signed_fields(packet))

    verify_signature(drone_sign_pub, signed_bytes, _b64d(packet["signature"]))

    if not verify_mac(mac_key, signed_bytes, _b64d(packet["mac"])):
        raise ValueError("MAC verification failed")

    message_id = f"{packet['drone_id']}:{packet['timestamp']}:{packet['nonce']}"
    if is_replay(message_id):
        raise ValueError("replay detected")
    mark_timestamp_used(message_id)
```

```
    plaintext = decrypt_data(enc_key, _b64d(packet["nonce"]),
_b64d(packet["ciphertext"]))
    return plaintext


def main():
    """Simulates secure Drone -> Ground Station communication.

    Features covered:
      - Password hashing+salting authentication
      - Diffie–Hellman key exchange (shared secret)
      - Hybrid encryption: RSA-OAEP to wrap AES session key, AES-CTR for data
      - Digital signature (RSA-PSS)
      - Integrity via HMAC-SHA256
      - Replay protection via timestamp (bound into MAC+signature)
    """

    reset_timestamps()
    print(" Secure Drone Communication Start\n")

    # ---------------- SERVER REGISTRATION (stores only salt+hash) ----------------
    drone_id = "DR001"
    password = "drone123"
    salt, stored_hash = hash_password(password)
    server_user_db = {drone_id: (salt, stored_hash)}

    # ---------------- AUTH (server verifies) ----------------
    salt_s, stored_hash_s = server_user_db[drone_id]
    if not verify_password(password, salt_s, stored_hash_s):
        print(" Authentication failed")
        return
    print(" Authentication successful")

    # ---------------- KEY EXCHANGE (DH) ----------------
    drone_priv, drone_pub, params = generate_dh_keys()
    server_priv = params.generate_private_key()
    server_pub = server_priv.public_key()

    dh_shared_drone = generate_shared_key(drone_priv, server_pub)
    dh_shared_server = generate_shared_key(server_priv, drone_pub)
    if dh_shared_drone != dh_shared_server:
        print(" Key exchange failed")
        return
    print(" Shared DH secret established")
    print("   DH secret fingerprint (SHA-256):",
hashlib.sha256(dh_shared_server).hexdigest())

    # ---------------- RSA KEYS ----------------
```

```
    # Drone uses RSA keypair for signing. Server uses RSA keypair for decrypting AES
session key.
    drone_sign_priv, drone_sign_pub = generate_rsa_keys()
    server_rsa_priv, server_rsa_pub = generate_rsa_keys()

    # --------------- DRONE PREPARES MESSAGE ---------------
    telemetry = {
        "drone_id": drone_id,
        "latitude": 12.97,
        "longitude": 77.59,
        "speed": 45,
    }

    aes_session_key = os.urandom(32)  # 256-bit AES session key
    enc_session_key = rsa_encrypt_key(server_rsa_pub, aes_session_key)
    print(" AES session key wrapped with RSA (bytes):", len(enc_session_key))

    enc_key_drone, mac_key_drone = derive_session_keys(dh_shared_drone,
aes_session_key)

    plaintext = json.dumps(telemetry, sort_keys=True, separators=(",", ":"))
    nonce, ciphertext = encrypt_data(enc_key_drone, plaintext)
    print(" Telemetry encrypted (ciphertext bytes):", len(ciphertext))

    timestamp = generate_timestamp()

    packet = {
        "drone_id": drone_id,
        "timestamp": timestamp,
        "nonce": _b64e(nonce),
        "ciphertext": _b64e(ciphertext),
        "enc_session_key": _b64e(enc_session_key),
    }

    signed_bytes = _canonical_bytes(_signed_fields(packet))
    packet["mac"] = _b64e(generate_mac(mac_key_drone, signed_bytes))
    packet["signature"] = _b64e(sign_message(drone_sign_priv, signed_bytes))

    print(" Data sent securely\n")

    # ================ SERVER SIDE ================
    try:
        decrypted = process_packet(
            packet,
            server_rsa_priv=server_rsa_priv,
            drone_sign_pub=drone_sign_pub,
            dh_shared_secret_server=dh_shared_server,
        )
```

```python
        print(" Signature verified")
        print(" MAC verified")
        print("\n Final Decrypted Data:")
        print(decrypted.decode("utf-8"))
    except Exception as e:
        print(f" Server rejected message: {e}")
        return

    # ---------------- BONUS: REPLAY ATTACK SIMULATION ----------------
    print("\n--- Simulating Replay Attack ---")
    try:
        _ = process_packet(
            packet,
            server_rsa_priv=server_rsa_priv,
            drone_sign_pub=drone_sign_pub,
            dh_shared_secret_server=dh_shared_server,
        )
        print(" Replay attack not detected!")
    except Exception as e:
        print(f" Replay attack detected and blocked: {e}")


if __name__ == "__main__":
    main()
```

PART 2 — Short Report

1. System Design

The system simulates secure bidirectional communication between a Drone (client) and a Ground Control Station (server). It is structured as six independent Python modules plus a main orchestration script, each handling one cryptographic concern.

Module	Responsibility	Algorithm
key_exchange.py	Shared secret negotiation	Diffie-Hellman 2048-bit + HKDF-SHA256
encryption.py	Data confidentiality + key wrapping	AES-256-CTR + RSA-2048-OAEP
authentication.py	Entity authentication	Password + Salt + SHA-256
signature.py	Non-repudiation	RSA-2048-PSS (SHA-256)
integrity.py	Message integrity	HMAC-SHA256
replay.py	Replay attack prevention	Timestamp window + Nonce deduplication

2. Explanation of Implementation

The workflow proceeds in five stages:

- Authentication: The drone proves its identity by providing a password. The server verifies it against a stored (salt, SHA-256 hash) pair — no plaintext passwords are ever stored.
- DH Key Exchange: Both parties generate 2048-bit DH keypairs, exchange public keys, and independently derive the same shared secret. HKDF stretches this into a 32-byte encryption key and a 32-byte MAC key.
- Hybrid Encryption: The drone generates a fresh 256-bit AES session key per session, encrypts it with the server's RSA public key (OAEP/SHA-256), then encrypts the telemetry JSON with AES-CTR using a random nonce.
- Signing & MAC: The drone signs a canonical byte representation of the packet using RSA-PSS, then computes an HMAC-SHA256 MAC over the same bytes using the derived MAC key. Both are base64-encoded and attached to the packet.
- Server Verification: The ground station checks the timestamp window, decrypts the AES session key, re-derives session keys, verifies the RSA-PSS signature, validates the HMAC, checks the nonce deduplication set, and finally decrypts the payload.

3. Module-by-Module Code Explanation

`key_exchange.py`

Implements Diffie-Hellman (DH) key exchange using 2048-bit parameters. Both drone and ground station generate DH keypairs, exchange public keys, and independently compute the same shared secret. HKDF (HMAC-based Key Derivation Function with SHA-256) then stretches this shared secret along with the AES session key into 64 bytes, yielding a 32-byte AES encryption key and a 32-byte MAC key. This ensures forward secrecy for each session.

`encryption.py`

Implements hybrid encryption. The drone generates a fresh 256-bit AES session key per session. This key is encrypted with the ground station's RSA-2048 public key using OAEP padding (SHA-256) — ensuring only the server can recover it. Payload telemetry data is then encrypted with AES in CTR mode using a random 16-byte nonce. CTR mode turns AES into a stream cipher, requiring no padding and allowing parallel operation.

`authentication.py`

Implements password-based authentication with hashing and salting. A 16-byte cryptographically random salt is generated using os.urandom(). The password is concatenated with the salt and hashed with SHA-256. Only the (salt, hash) pair is stored — never the plaintext password. On verification, the same salt is used to recompute the hash and compared in constant time.

`signature.py`

Implements RSA digital signatures using PSS (Probabilistic Signature Scheme) padding with SHA-256. The drone signs a canonical JSON representation of the packet fields (drone_id, timestamp, nonce, ciphertext, enc_session_key). The ground station verifies the signature using the drone's public key. PSS is probabilistic and provides stronger security guarantees than older PKCS#1 v1.5 padding.

`integrity.py`

Generates and verifies HMAC-SHA256 MACs over the signed packet fields. HMAC uses the derived mac_key (from HKDF) as the secret. hmac.compare_digest() is used for verification to prevent timing side-channel attacks. The MAC covers the same canonical bytes as the digital signature, providing a second independent integrity check.

`replay.py`

Prevents replay attacks using a dual mechanism: timestamp window checking (rejects messages older than 30 seconds) combined with a nonce-based deduplication set. Each accepted message's composite ID (drone_id + timestamp + nonce) is added to a seen-set. Any retransmission of the exact same packet is detected and rejected, even within the 30-second window.

`main.py`

Orchestrates the full secure communication workflow: (1) Drone authenticates with the ground station via password hash verification. (2) DH key exchange establishes a shared secret. (3) Drone generates an AES session key, wraps it with RSA-OAEP, encrypts telemetry with AES-CTR, signs the packet with RSA-PSS, and attaches an HMAC-SHA256 MAC plus timestamp. (4) Ground station decrypts, verifies signature and MAC, checks replay protection, and decrypts the payload. The bonus section replays the same packet to demonstrate detection.

4. Security Analysis

Attacks Prevented:

- Eavesdropping / Sniffing: AES-256-CTR encryption ensures that even if packets are intercepted, the payload remains confidential.
- Man-in-the-Middle (MITM): RSA digital signatures (PSS) bind the message to the drone's private key. Any tampering invalidates the signature, and the HMAC provides a second layer of integrity verification.
- Replay Attacks: The dual mechanism of a 30-second timestamp window and a nonce deduplication set ensures that replayed packets are always rejected, even within the valid time window.
- Password Theft: Passwords are never stored in plaintext. The SHA-256 + random salt approach ensures that even a database breach does not expose raw passwords.
- Key Compromise (Partial): Because a fresh AES session key is generated for each session, compromise of one session key does not expose any other session (forward secrecy at the session level).

Limitations:

- Public Key Distribution: The system currently assumes that public keys (RSA, DH parameters) are pre-shared securely. In a real deployment, a PKI (Public Key Infrastructure) or certificate authority would be required to prevent impersonation during the DH exchange.
- No Perfect Forward Secrecy for RSA: If the server's RSA private key is compromised, previously recorded sessions can be decrypted. Switching to ECDH ephemeral keys would fully address this.
- In-Memory Nonce Store: The replay protection set (used_message_ids) is in-memory only and is lost on restart. A persistent store (e.g., Redis or a database) would be required in production.
- Password Hashing: SHA-256 is used for password hashing. Production systems should use a memory-hard KDF such as bcrypt, scrypt, or Argon2id, which are more resistant to GPU-based brute-force attacks.